Qoder Exam Playbook

9 hours · 3000 credits · prompts only · marked by cloning and running locally

1. Which model for what

Job	Model	Rate	Why
Planning: brief PDF to SPEC.md	Ultimate (or Qwen3.8-Max / DeepSeek-V4-Pro to save)	~2.0x (0.5x)	Spec quality decides everything. Cheapest place to fix mistakes.
Building features (most of the exam)	Efficient, or a Flash model (Qwen3.8-Flash, DeepSeek-Flash)	0.3x / 0.1x	The spec already did the thinking. Cheap models handle well-defined tasks.
Tricky feature / non-obvious bug	Qwen3.8-Max or DeepSeek-V4-Pro	0.5x	Step up only when a cheap model struggles.
Stuck after 2 failed fixes	Ultimate, once, in a new chat	~2.0x	One strong attempt beats looping on a cheap model.
Final marker check	Strong model (0.5x Pro or Ultimate)	0.5-2.0x	Needs to reason across the whole codebase against the requirements.
AVOID	Auto	varies	Tends to route trivial tasks to expensive models. Pick deliberately.

Rates change. Check the number shown in the model selector on the day. Skip Quest's Experts (multi-agent) mode unless you're well ahead.

2. Setup (first 10 minutes)

- Create the project folder and open it in Qoder.
- Add rules: **Settings > Rules**, add a project rule set to always apply, paste **qoder-rules.md**. Or create the file **.qoder/rules/project.md**.
- Make sure your designprompts.dev style is pasted into the UI style section of the rules.
- Keep the rules lean. They're sent with every request.
- **Quest vs Editor:** same app (Open Quest button, top right). Quest 'Code with Spec' for planning; Editor agent chat for build chunks and fixes. If only one: Quest, but feed it 1-3 tasks at a time.

3. The workflow

Phase	Time	What you do
Plan	~45 min	Strong model + Planning prompt + brief PDF attached. Then READ SPEC.md yourself: missing requirements? wrong assumptions? Fix them now.
Build	~6 h	New chat per 1-3 tasks. Cheap model + Build prompt. Test in the browser. git commit. Repeat. MUST features before NICE-TO-HAVE.
Final	~1.5 h	Strong model + Final check prompt. Fix gaps, finish README, run setup from a fresh clone exactly like a marker would.

4. When to start a new chat

- After finishing 1-3 tasks and committing (natural break point).
- When moving to a different area (e.g. auth to dashboard).
- Around 10-15 back-and-forths in one chat.
- When the model forgets rules or repeats mistakes.
- You lose nothing: SPEC.md and PROGRESS.md carry the state. Old context keeps costing credits even when irrelevant.

5. Bug fixes

- **Small and obvious** (alignment, typo, wrong label): one line, no template.
- **Error message or unclear cause:** use the Bug-fix prompt and paste the exact error.

- **Fails twice:** stop. git revert if it got messier, new chat, stronger model, one attempt.

6. Parallel or sequential?

- **Sequential by default.** Parallel agents overwrite shared files (layout, schema), burn credits twice as fast, and are hard to verify.
- While the agent works: test the last feature, re-read the brief, write the next prompt, commit.
- Parallel only for truly separate files (e.g. README vs a standalone page), and only after T1 (scaffold + schema) is done.

7. Prompt quick reference (full text in exam-prompts.md)

Prompt	Model	Use
1. Planning	Strong	Brief to SPEC.md: requirements R1..Rn, assumptions, data model, routes, tasks T1..Tn, marker test plan. No code.
2. Build	Cheap	"Implement T_ to T_ from SPEC.md, following the project rules..."
3. Bug-fix	Cheap; step up if stuck	What you did / what happened / what you expected + exact error. Root cause only.
4. Final check	Strong	Act as the marker: verify every R, fix gaps, README, fresh-clone test.

8. Before you submit

- Fresh clone, then **npm install** and **npm run dev** work with no manual steps.
- App works on an empty DB. Seed is optional (npm run seed) unless the brief says otherwise.
- .env.example committed with working defaults. No keys required.
- README: features mapped to the brief, exact run steps, Node version, demo accounts, limitations.
- Every requirement in SPEC.md ticked and actually tested in the browser.